Intermediate (test a tool without calling the LLM) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

This exercise separates two questions that get blurred together if you only ever test through the full agent:

1. Does the tool itself do the right thing, given specific arguments?
2. Does the model choose to call it, with the right arguments, at the right time?

Question 1 has nothing to do with the LLM, so it gets a normal, fast, exact pytest test — no API key, no network, no cost, no flakiness. Question 2 is a separate, harder problem (closer to the Real-world exercise later in this document).

A tool test isn't only "does it work with good input." A real model sometimes calls a tool in a way that fails, or with arguments that make no sense. So also test: what does the tool return when it fails, and what happens when the arguments have the wrong type?

The exact pieces:

- Define the tools (`add` , `flaky_lookup`) inline in the practice file, so the test runs on its own with no other files.
- `from tools import add, flaky_lookup` — `add` is pure math, and `flaky_lookup` has a `should_fail` switch built in for exactly this kind of test.
- `flaky_lookup.invoke({"query": "x", "should_fail": True})` — the tool catches its own error and returns an `"Error: ..."` string the model can read, instead of crashing the agent.
- `with pytest.raises(ValidationError):` — checks that a block of code raises that error. `@tool` checks the arguments against the function's type hints (with Pydantic, like Doc04/06), so `add.invoke({"a": "two", "b": 3})` raises `ValidationError` before `add` even runs. Import it with `from pydantic import ValidationError` .

- Skip `get_weather` here: it calls a real weather API over the network, so it isn't a no-network test.

Hint 2 — The plan, and almost the whole thing

```
from tools import add, flaky_lookup

define test_add_returns_the_sum():     a couple of exact sums
define test_flaky_lookup_success():   no should_fail -> "Result for ..."
define test_flaky_lookup_failure():   should_fail=True -> "Error: ..."
define test_add_rejects_wrong_type():
    with pytest.raises(ValidationError):
        add.invoke with a = "two"
```

Here's most of it — write the failure test yourself:

```
# practice/tool_test_no_llm_practice.py
import pytest
from pydantic import ValidationError
from tools import add, flaky_lookup

def test_add_returns_the_sum():
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    # your turn: call it with should_fail=True, and check the exact
    # "Error: ..." string it returns instead of crashing
    ...

def test_add_rejects_wrong_type():
    with pytest.raises(ValidationError):
        add.invoke({"a": "two", "b": 3})
```

Full solution: Show me the solution